IMPLEMENTASI ALGORITMA GREEDY BERBASIS PROFIT DENSITY UNTUK OPTIMASI PEMILIHAN VARIETAS BIBIT PEPAYA CALIFORNIA & CALIFORNIA (IPB9)

"Laporan ini dibuat untuk memenuhi tugas akhir mata kuliah Desain Analisis Algoritma (B)"

Dosen Pengampu: Dyah Sulistyowati R, S.Kom., M.Kom.

Nama:

Nayunda Krisna Abi Pasya / 4524210077

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS PANCASILA

2025

ALGORITMA

Penjelasan

Algoritma yang digunakan adalah Greedy Algorithm. Algoritma ini memecahkan masalah langkah demi langkah dengan selalu memilih pilihan yang paling menguntungkan pada saat itu (lokal optimal) tanpa mempertimbangkan pilihan di masa depan. Dalam kasus ini, kita menggunakan strategi *Greedy by Density*, yaitu memprioritaskan bibit dengan nilai keuntungan tertinggi per unit lahan.

Pseudocode

1. Mulai
2. Input Daftar_Bibit dan Kapasitas_Lahan
3. Hitung Rasio = Profit / Luas_Lahan untuk setiap jenis bibit
4. Urutkan Daftar_Bibit berdasarkan Rasio secara Descending
5. Untuk setiap Bibit dalam Daftar_Bibit:
6. Jika Kapasitas_Lahan >= Luas_Lahan_Bibit:
7. Ambil_Bibit = Kapasitas_Lahan / Luas_Lahan_Bibit
8. Sisa_Lahan = Kapasitas_Lahan % Luas_Lahan_Bibit
9. Total_Profit = Total_Profit + (Ambil_Bibit * Profit_Bibit)
10. Tampilkan Total_Profit dan Rincian_Bibit
11. Selesai

NP-PROBLEM & STUDI KASUS

Penjelasan

Masalah pemilihan bibit ini merupakan turunan dari Knapsack Problem, khususnya jenis *Unbounded Knapsack* karena jumlah tiap bibit tidak dibatasi selama lahan tersedia. Masalah ini termasuk dalam kategori NP-Hard karena untuk mencari kombinasi paling absolut (optimal global) pada jumlah data yang masif, waktu yang dibutuhkan akan meningkat secara eksponensial. Algoritma Greedy digunakan sebagai pendekatan untuk mendapatkan solusi yang mendekati sempurna dalam waktu yang sangat cepat.

Studi Kasus

Seorang agen bibit memiliki lahan distribusi seluas 120 unit. Terdapat dua jenis bibit yang tersedia:

1. **California:** Keuntungan Rp15.000/bibit, membutuhkan lahan 2 unit.
2. **Calina (IPB9):** Keuntungan Rp25.000/bibit, membutuhkan lahan 3 unit.

ANALISIS KOMPLEKSITAS

Time Complexity: $O(n \log n)$. Kompleksitas ini didominasi oleh algoritma *sorting* (pengurutan) bibit berdasarkan rasio. Jika bibit hanya ada 2 jenis, proses ini sangat cepat, namun secara teoritis pada n data adalah**:** $O(n \log n)$.

Space Complexity: $O(n)$. Membutuhkan ruang memori untuk menyimpan daftar data bibit dan hasil perhitungan.

PENYELESAIAN

Seorang agen bibit memiliki lahan distribusi seluas 120 unit. Terdapat dua jenis bibit yang tersedia:

1. California: Keuntungan Rp15.000/bibit, membutuhkan lahan 2 unit.
2. Calina (IPB9): Keuntungan Rp25.000/bibit, membutuhkan lahan 3 unit.

PROGRAM

Link Google Colab

https://colab.research.google.com/drive/1bYujXfwVTla4YdVinbIgbfb3WzTby_uh?usp=sharing

Program

```
def solusi_optimasi_bibit(kapasitas, data):
    data.sort(key=lambda x: x['profit']/x['lahan'], reverse=True)

    total_profit = 0
    hasil_pilihan = []
    sisa_lahan = kapasitas

    for b in data:
        if sisa_lahan >= b['lahan']:
           jumlah = sisa_lahan // b['lahan']
           total_profit += jumlah * b['profit']
           sisa_lahan -= jumlah * b['lahan']
           hasil_pilihan.append((b['nama'], jumlah))

    return hasil_pilihan, total_profit, sisa_lahan

kapasitas_lahan = 120
bibit_tersedia = [
    {"nama": "California", "profit": 15000, "lahan": 2},
    {"nama": "Calina (IPB9)", "profit": 25000, "lahan": 3}
]

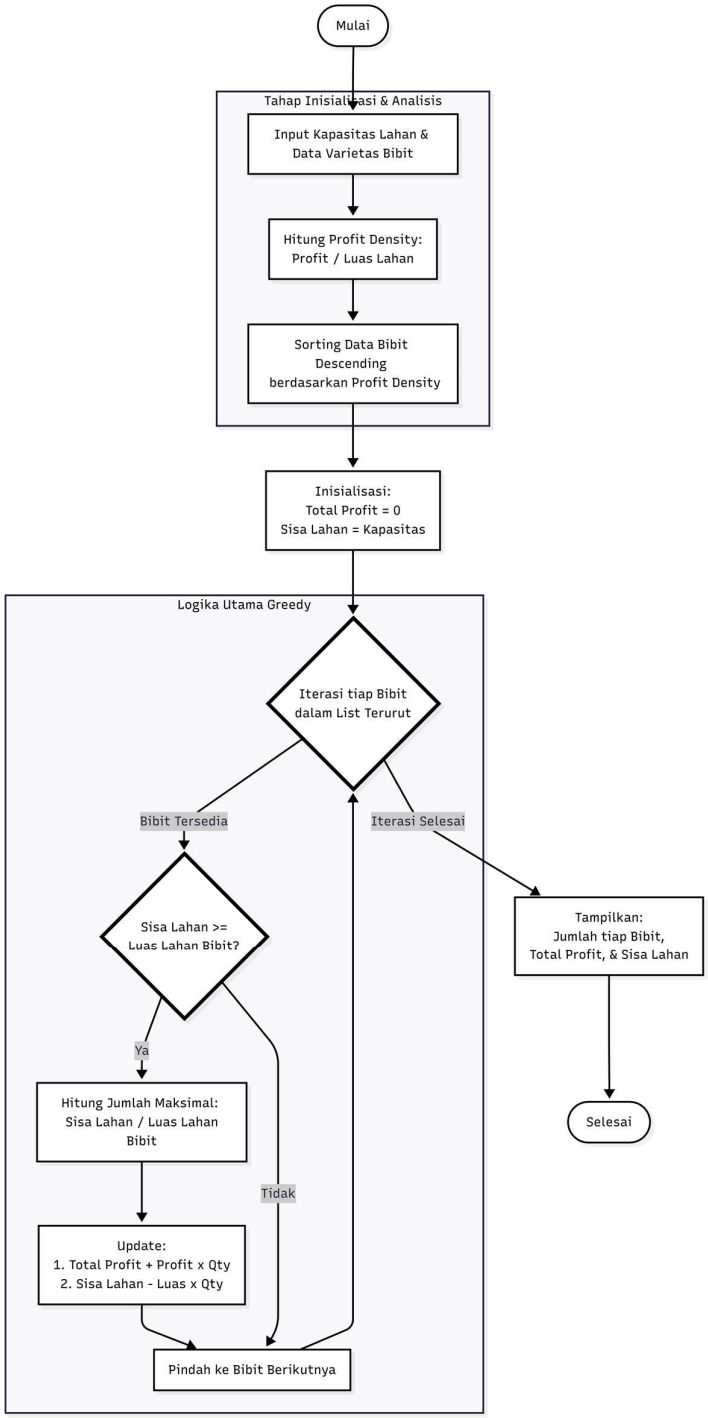
hasil, profit, sisa = solusi_optimasi_bibit(kapasitas_lahan,
bibit_tersedia)

print(f"Hasil Optimasi Penjualan:")
for nama, qty in hasil:
    print(f"- {nama}: {qty} bibit")
print(f"Total Profit: Rp{profit}")
print(f"Sisa Lahan: {sisa} unit")
```

Output

```
Hasil Optimasi Penjualan:
- Calina (IPB9): 40 bibit
Total Profit: Rp1000000
Sisa Lahan: 0 unit
```

Flowchart



KESIMPULAN

Algoritma Greedy memberikan solusi yang efisien untuk masalah penentuan stok bibit pepaya. Dengan memprioritaskan bibit Calina (IPB9) yang memiliki densitas profit lebih besar, distributor dapat memaksimalkan keuntungan pada lahan yang terbatas. Secara komputasi, algoritma ini sangat handal karena memiliki kompleksitas waktu yang rendah.